

Capstone Review Questions - Short Speaking Answers

Concise 1–3 sentence rehearsal version

1. What problem does your project solve?

Answer: It gives teams one place to manage users, projects, tasks, and status. It replaces scattered updates with a structured, trackable workflow.

2. What were the main objectives of your project?

Answer: Build a secure full-stack app with real CRUD flows, role-based access, and useful dashboard visibility. Also make it testable and easy to run in dev/prod.

3. Explain your application's architecture.

Answer: React frontend, Express/Node backend, MongoDB database. React talks to REST APIs, backend enforces rules, Mongo stores the data.

4. Walk us through a request from React to MongoDB and back.

Answer: User action in React sends an API call. Express middleware validates/authenticates, backend updates MongoDB, then returns JSON for React to render.

5. What is the role of React in your application?

Answer: React handles UI, routing, forms, and state updates. It's the layer users interact with directly.

6. Why did you use Node.js and Express?

Answer: It keeps the stack consistent with JavaScript/TypeScript. Express makes API routes and middleware straightforward.

7. What is MongoDB's role?

Answer: MongoDB persists the app data—users, projects, tasks, and audit logs. It's the long-term storage layer.

8. What happens when a user logs in?

Answer: Credentials go to the login API, backend verifies them, and returns a JWT plus user info. Frontend stores that and uses it for protected calls.

9. What is JWT and why did you use it?

Answer: JWT is a signed auth token. I used it for stateless API authentication and protected endpoint access.

10. What happens if an unauthenticated user calls a protected API?

Answer: Auth middleware blocks it and returns 401. The route logic never runs.

11. What middleware does your application use?

Answer: JSON parsing, CORS, auth, role authorization, validation, error handling, and audit logging middleware.

12. Explain one CRUD operation end to end.

Answer: Creating a task: React submits a POST request, backend validates and saves to MongoDB, API returns created task, UI refreshes.

13. What HTTP methods did you use?

Answer: GET, POST, PUT/PATCH, and DELETE. Standard REST semantics across resources.

14. What HTTP status codes should your application return?

Answer: 200/201/204 for success, 400 for bad input, 401/403 for auth issues, 404 for missing resources, 500 for server errors.

15. How did you validate user input?

Answer: Primarily on the backend with validators and schema rules. Frontend validation helps UX but backend is the enforcement point.

16. How do you handle errors?

Answer: Backend returns structured errors with proper status codes. Frontend catches failures and shows meaningful messages.

17. How did you test your APIs?

Answer: I used automated integration/unit tests and manual endpoint verification. I also validated behavior through UI workflows.

18. How can you prove that data is actually being stored in MongoDB?

Answer: Create/update from the app, then inspect the Mongo collection and show the matching document.

19. How did Git help your team?

Answer: Feature branches, commit history, PR review, and safer merges. It made collaboration and rollback much easier.

20. What does npm do in your project?

Answer: Manages dependencies and scripts for install, run, build, test, and seed tasks.

21. What was one significant technical challenge?

Answer: Resolving feature-branch merge conflicts without breaking tests. I fixed conflicts, reran tests, and revalidated environments.

22. How did your team troubleshoot problems?

Answer: We debugged layer by layer: frontend request, network response, backend logs, then database state.

23. What did you know before the project compared with what you know now?

Answer: Before, I knew tools separately. Now I can build and explain a full integrated system end to end.

24. What is the most important thing you learned from the project?

Answer: How to build for reliability: clear contracts, test coverage, and repeatable operational workflows.

25. What part of the project best demonstrates that you applied what you learned?

Answer: The admin + audit flow. It combines frontend, backend, auth, DB, testing, and merge-safe integration.

26. Why should we believe this is more than just code that happens to work?

Answer: Because I can explain architecture decisions, demonstrate data flow, show tests, and reproduce behavior live.

27. If you had another week, what would you improve?

Answer: More automated regression tests, stronger observability, and polish on audit analytics UI.

28. What would happen if MongoDB became unavailable?

Answer: DB operations fail, backend returns controlled errors, and health checks/logs should make that visible quickly.

29. Why shouldn't React communicate directly with MongoDB?

Answer: You'd bypass auth, authorization, validation, and business rules. The backend is the security boundary.

30. Can you explain your entire project in 30 seconds?

Answer: I built a full-stack task tracker with React, Express, and MongoDB, secured by JWT and roles, plus admin and audit capabilities, with tests and containerized workflows.

31. What did we build?

Answer: A role-based project/task tracking platform with admin controls and audit visibility.

32. How does it work?

Answer: Users log in, use React UI, and every action goes through secured APIs to MongoDB.

33. What did I personally contribute?

Answer: Backend middleware/routes, audit integration, conflict resolution, and validation via rebuild/reseed/tests.

34. What problem did we encounter and solve?

Answer: We hit merge conflicts and post-merge regressions. I resolved them and confirmed clean merges and passing tests.

35. What can I do now that I could not do before this training/project?

Answer: I can design, build, secure, test, deploy, and explain a real full-stack app from end to end.